UNIVERSITI KEBANGSAAN MALAYSIA
The National University of Malaysia

FAKULTI TEKNOLOGI & SAINS MAKLUMAT

TTTC3413
ROBOT APPLICATIONS

Topic:

Autonomous Crack Detection Robot for Industrial Infrastructure Inspection

Lecturer's Name:
Dr. Anahita Ghazvini

Num.	Name	No. Matriks Number
1	NANCY ANNE	A202459
2	KAVI PRIYA A/P BALASUPRAMANIAM	A202660

ISI KANDUNGAN

1.0 Introduction

The rapid evolution of Industry 4.0 has encouraged multiple sectors to adopt autonomous technologies and artificial intelligence to enhance operational efficiency and safety. One area greatly impacted by these advancements is the inspection of pavements and industrial surfaces for structural cracks. Traditionally, crack detection relies heavily on manual visual examination by human inspectors, a method that is slow, inconsistent and highly vulnerable to environmental conditions such as poor lighting, shadows and surface irregularities. These limitations often lead to missed defects and delayed maintenance, which can compromise structural safety over time (Fan et al., 2022). Furthermore, human-based inspection exposes workers to hazardous areas such as industrial floors, busy roadways or unstable surfaces, making the process risky and inefficient (Nguyen et al., 2022).

To address these issues, autonomous robotic systems have become a preferred alternative due to their ability to perform high-precision inspection tasks consistently and safely. Deep learning-based computer vision models, particularly models such as YOLOv8, have shown remarkable performance in detecting fine cracks across complex environments with high accuracy and robustness (Liu et al., 2023). When integrated into a mobile robotic platform, these capabilities enable continuous, automated inspection that reduces human involvement in dangerous environments while improving defect detection rates.

This project develops an autonomous crack detection robot using the TurtleBot3 Waffle Pi, a lightweight mobile robot widely used in research and educational robotics due to its modular design and strong compatibility with ROS 1 Noetic (Huang et al., 2022). The system combines a YOLOv8 deep learning model trained on a diverse crack dataset with a ROS-based navigation and camera pipeline, enabling the robot to analyse surfaces in real time while moving autonomously. Simulation plays a critical role in the development pipeline, as Gazebo allows the team to evaluate robot behaviour, camera response and crack detection performance under controlled conditions before real-world deployment. According to Kaveh et al. (2024), simulation-based testing is essential for validating advanced inspection algorithms due to the high cost and risk associated with on-site testing.

Throughout development, tools such as Google Colab, Anaconda and OpenCV are used to support model training, real-time inference and image preprocessing tasks. These technologies ensure that the inspection process is accurate, reliable and efficient. The integration between simulation and real implementation is clearly illustrated in Figure 1, which shows the TurtleBot3 Waffle Pi in both its physical form and its virtual representation within the Gazebo environment. This alignment between simulation and real-world deployment forms the foundation of a complete automated inspection pipeline.

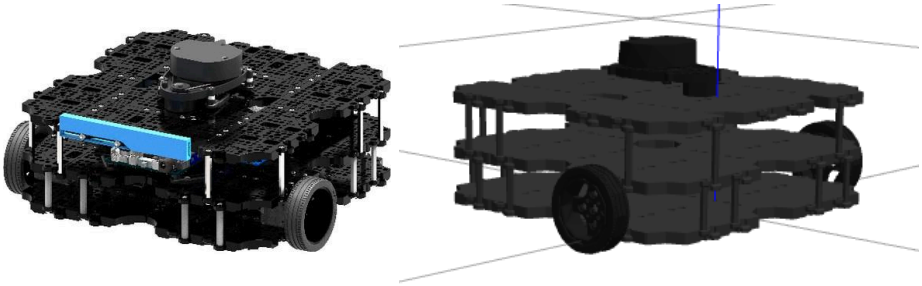

Figure 1: Robot TurtleBot3 Waffle Pi in real-world and simulation environments

2.0 Objectives

The objectives of this project are designed to ensure the successful development of a complete autonomous crack detection system integrating deep learning, robotics and simulation-based validation. These objectives reflect the needs identified in recent studies, where deep learning models and mobile robots have demonstrated great potential for accurate structural inspection and automated maintenance workflows (Fan et al., 2022; Zhang et al., 2024). Each objective contributes to creating a system that is reliable, efficient and suitable for real-world deployment in industrial environments.

1. Development of a mobile robotic inspection platform
   The first objective is to design and develop a fully functioning mobile inspection robot using the TurtleBot3 Waffle Pi operating under ROS 1 Noetic. This robot serves as the autonomous platform responsible for navigation, camera data collection and real-time movement control, consistent with recommendations by Huang et al. (2022) regarding ROS-based robotic inspection solutions.

2. Training an accurate YOLOv8 crack detection model
   The second objective is to train a high-performance YOLOv8 deep learning model capable of identifying various pavement crack types with strong precision and recall. This objective follows the findings of Liu et al. (2023) who highlight the superiority of modern deep learning approaches for fine-scale crack detection.

3. Implementing real-time detection using anaconda and webcam input
   The third objective is to deploy the trained YOLOv8 model within an Anaconda environment to test real-time crack detection using a standard webcam. This ensures that the system performs reliably under real lighting variations and dynamic camera movements, a challenge commonly reported in industrial inspection research (Nguyen et al., 2022).

4. Constructing and validating a gazebo simulation environment
   The fourth objective is to build a realistic Gazebo simulation world that includes pavement textures, obstacles and road environments. This allows the crack detection model and robot navigation system to be evaluated safely and repeatedly, which aligns with Kaveh et al. (2024) who emphasize the importance of simulation for autonomous inspection tasks.

5. Integrating navigation, deep learning and simulation into one system
   The final objective is to integrate the navigation pipeline, deep learning crack detection model and simulation components into a unified autonomous system. This integration demonstrates the feasibility of robotics and AI for automated infrastructure inspection, a direction strongly supported by recent advancements in industrial robotics applications (Zhang et al., 2024).

3.0 Problem Statement

The main challenges faced in crack detection for pavements, industrial floors and public infrastructure are:

1. Inaccurate and inconsistent manual crack detection
   Traditional visual inspection often fails to identify subtle or hairline cracks due to human fatigue, inconsistent lighting, shadows and uneven surface textures. These

factors greatly reduce accuracy and lead to overlooked structural defects (Nguyen et al., 2022).

2. Time-consuming and impractical for large-scale inspection
   Manual inspection requires significant time and manpower, making it ineffective for large industrial environments or long roadway networks. Delays in detecting cracks increase the risk of structural deterioration and maintenance failures (Fan et al., 2022).

3. Safety risks for human inspectors
   Inspectors frequently work in hazardous locations such as active industrial zones, slippery surfaces, high-traffic roads or unstable floors placing them at risk of injury. Many of these hazards could be avoided through automation.

4. Technical challenges in developing a fully autonomous crack-inspection robot
   Although deep learning models such as YOLOv8 are highly effective at detecting cracks (Liu et al., 2023), integrating them into an autonomous robotic system is complex. Real-time processing, mobile robot navigation, obstacle avoidance, sensor integration and simulation testing must all operate cohesively. Synchronising robot movement, camera perception and deep learning inference introduces additional engineering difficulty (Kaveh et al., 2024).

4.0 Solution Approach

To overcome the challenges identified, this project introduces a complete autonomous crack detection system that integrates deep learning, mobile robotics and simulation-based validation. Each component of the solution directly addresses a specific problem highlighted earlier.

4.1 Training YOLOv8 in Google Colab

One of the key problems with manual inspection is its inconsistent accuracy. To address this, the YOLOv8 deep learning model was trained on a diverse and augmented crack dataset using GPU acceleration on Google Colab. This training process enables the system to automatically learn various crack patterns, including fine, hairline and irregular cracks that human inspectors often miss. The

use of Colab ensures efficient and robust model training, resulting in a reliable and repeatable method for detecting cracks with high precision that directly addresses the human limitations identified earlier.

4.2 Real-Time Detection Using Webcam and Anaconda Prompt (RetinaNet)

webcam.py

File Edit View

```
import cv2
from ultralytics import YOLO

# Load the trained YOLOv8 model
model = YOLO('C:/YOLOModels/crack_detection_test/best.pt')

# Open webcam
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        print("Failed to grab frame")
        break

    # Run detection
    results = model(frame)

    # Annotate frame
    annotated_frame = results[0].plot()

    # Show webcam window
    cv2.imshow("Crack Detection", annotated_frame)

    # Press 'q' to quit
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Figure 2: Real-Time Crack Detection Using webcam.py in Anaconda Prompt

Inconsistent lighting and real-world environmental interference are major factors that contribute to missed defects during manual inspection. To address this, the trained YOLOv8 model was tested in an Anaconda environment using a real-time webcam feed. Figure 2 shows the webcam.py script executed from Anaconda Prompt (RetinaNet), which captures the live testing setup. This validation demonstrates the model's ability to handle varying lighting conditions, camera angles and surface textures. By testing on live video, the system effectively

mitigates visual inconsistencies that previously hindered human inspectors, confirming its practical reliability.

4.3 Gazebo Simulation of TurtleBot3

Manual inspection exposes workers to dangerous environments. To eliminate these risks, a Gazebo simulation was developed featuring cracked pavements, environmental obstacles and a simulated TurtleBot3 Waffle Pi. This simulation replicates real-world inspection scenarios without exposing anyone to physical danger. It also enables rigorous testing of robot navigation, obstacle avoidance and crack detection before deploying the robot in physical environments, ensuring both safety and reliability.

4.4 Integration of ROS 1, TurtleBot3 and YOLOv8

A major limitation of manual inspection is the need for continuous human involvement. This system integrates YOLOv8 into ROS 1 Noetic, enabling the TurtleBot3 Waffle Pi to autonomously navigate and detect cracks in real time. Using ROS, the robot can subscribe to camera topics, process images, run inference and label cracks continuously without human intervention. This provides a consistent, tireless and fully autonomous inspection mechanism, overcoming the labour-intensive and inefficient nature of manual inspection.

5.0 Required Software and Tools

This project relies on a comprehensive set of software tools and frameworks to support development, training, simulation and deployment. Ubuntu 20.04 serves as the primary operating system due to its compatibility with ROS 1 Noetic, the middleware used for robot control, sensor integration and autonomous navigation. Gazebo is integrated with ROS to simulate robot behaviour, while RViz is used for visualizing data such as camera feeds, robot poses and detection overlays.

Python 3 is the main programming language, supported by libraries such as OpenCV for image preprocessing and PyTorch for deep learning tasks. YOLOv8, implemented through the Ultralytics framework, provides the core crack detection capability. Anaconda and Jupyter Notebook offer flexible development environments that

simplify experimentation. The TurtleBot3 Waffle Pi acts as the hardware platform for real-world testing, consistent with ROS-based robotic inspection studies (Huang et al., 2022; Zhang et al., 2024).

6.0 Data Distribution

The dataset used for this project is the UAPD Pavement Distress Dataset, which originally contains 3151 images. After cleaning and removing unsuitable or corrupted images, the dataset was reduced to a total of 1025 images. To ensure efficient learning and validation, the dataset was split as follows:

- Training Set (80%): 820 images
- Validation Set (20%): 205 images

This cleaned and split dataset provides a reliable basis for training the YOLOv8 model while maintaining an appropriate balance for validation.

7.0 Methodology

7.1 Data Collection and Preprocessing

The images and their corresponding annotation and label files were first downloaded into Google Drive and then mounted in Google Colab to facilitate seamless access for training the YOLOv8 model. A custom Python script was used to organize the dataset by safely shuffling the image files and splitting them into training and validation sets with an 80:20 ratio. During this process, each image was paired with its respective annotation and label files to ensure consistency and data integrity. The script also created separate folder structures for the training and validation subsets, including dedicated subfolders for images, annotations and labels, enabling an organized workflow for robust model training and evaluation.

7.2 Annotation Conversion from XML to TXT

```
    size = root.find('size')
    w = int(size.find('width').text)
    h = int(size.find('height').text)

    out_file = os.path.join(labels_dir, os.path.splitext(xml_file)[0] + '.txt')
    with open(out_file, 'w') as f:
        for obj in root.iter('object'):
            cls = obj.find('name').text
            if cls not in classes:
                continue
            cls_id = classes.index(cls)
            xmlbox = obj.find('bndbox')
            b = (
                int(xmlbox.find('xmin').text),
                int(xmlbox.find('xmax').text),
                int(xmlbox.find('ymin').text),
                int(xmlbox.find('ymax').text)
            )
            bb = convert_bbox((w, h), b)
            f.write(f"{cls_id} {' '.join([str(a) for a in bb])}\n")
```

Figure 3: Annotation Conversion Script

YOLOv8 requires bounding box annotations in TXT format rather than XML. Therefore, an annotation conversion script was developed to automatically convert XML files into normalized TXT labels. As illustrated in Figure 3, this script ensures that every bounding box is converted according to YOLOv8's format, including class ID and normalized coordinates for x-center, y-center, width and height. This conversion step is crucial to maintaining annotation consistency across datasets.

7.3 Model Training

```
from ultralytics import YOLO

# Load YOLOv8 small model pretrained on COCO
model = YOLO('yolov8s.pt')

# Train the model
model.train(
    data='/content/drive/MyDrive/crack-detection/data.yaml',  # YAML config
    epochs=50,      # Number of training epochs
    imgsz=512,      # Resize images to 512x512 for training
    batch=8,        # Batch size
    project='/content/drive/MyDrive/crack-detection/crack_detection',  # Save results in Drive
    name='yolov8_crack',
    exist_ok=True
)
```

Figure 4: YOLOv8 Training Initialization

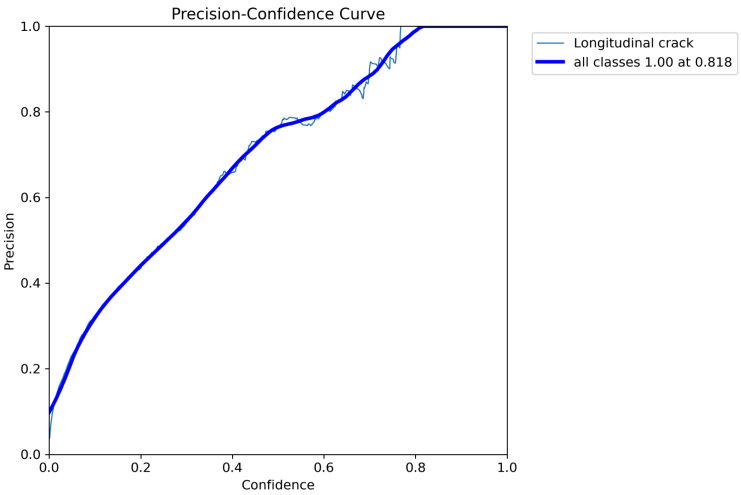

Figure 5: High Precision at High Confidence

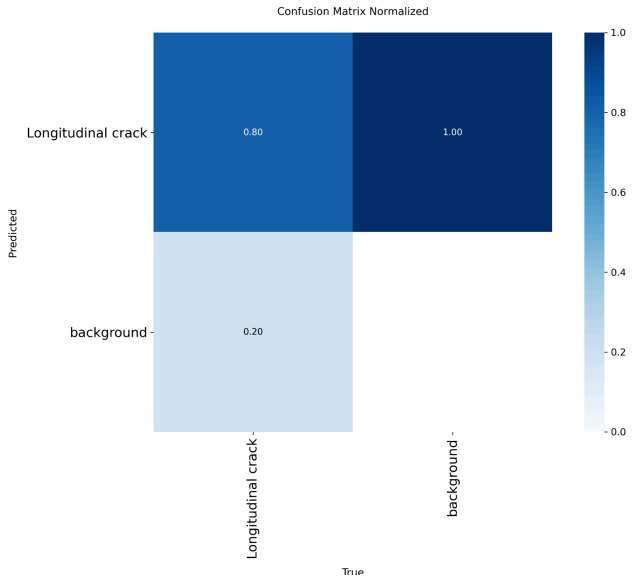

Figure 6: Normalized Confusion Matrix

Training was performed using transfer learning by initializing the model with pretrained YOLOv8s weights, following best practices outlined by Liu et al. (2023). The training script, shown in Figure 4, configures hyperparameters such as learning rate, batch size and the number of epochs (50). The model achieved high performance in crack detection as shown in Figure 5, it reaches 100% precision for all crack classes when the confidence threshold is set above approximately 0.82, confirming minimal false positives for the most certain detections. Additionally, as illustrated in Figure 6, the normalized confusion

matrix shows that the model achieves an 80% recall (0.80 true positive rate) for crack detection, demonstrating strong effectiveness in identifying existing defects which is crucial for infrastructure inspection.

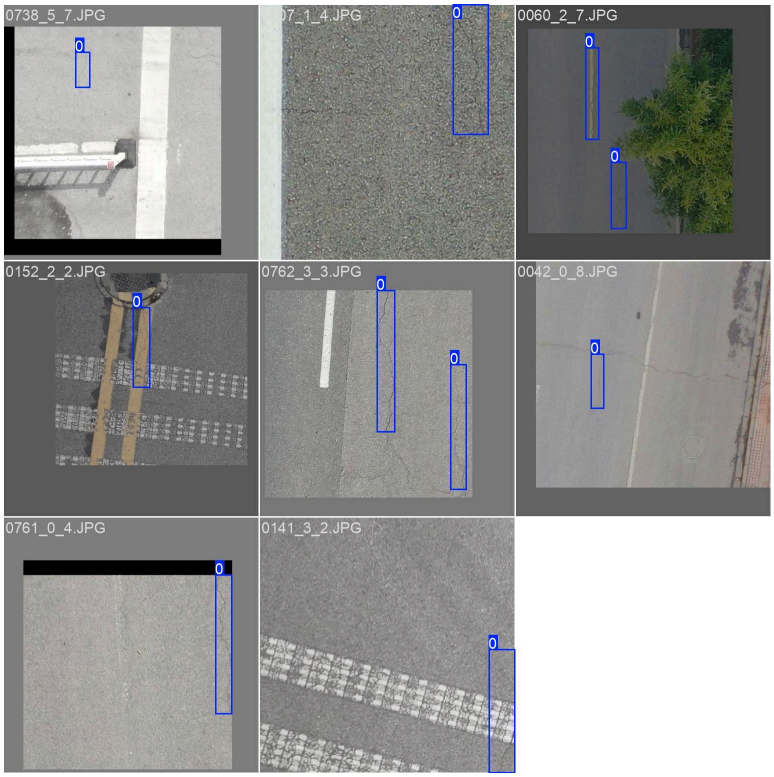

Figure 7: Training Batch Sample

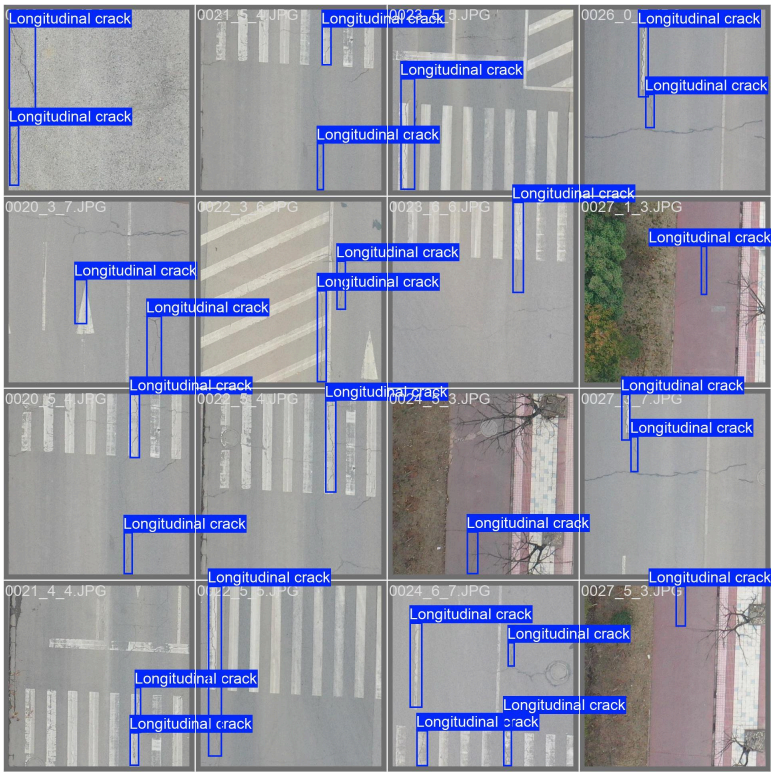

Figure 8: Ground Truth (Validation Batch)

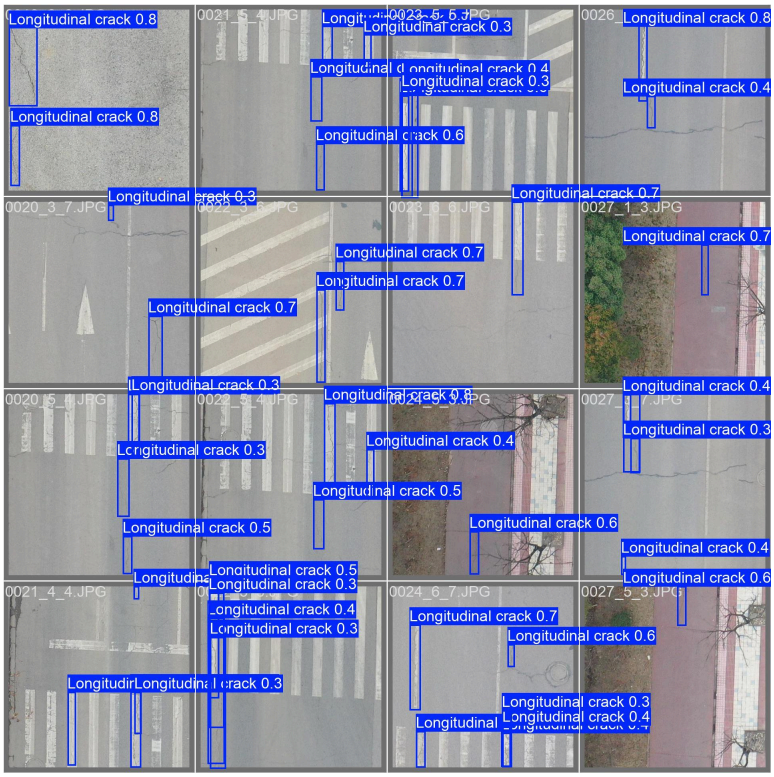

Figure 9: Model Prediction (Validation Batch)

These metrics demonstrate that the model successfully identifies cracks with minimal false positives while maintaining strong sensitivity to true crack instances. Figure 7 shows a sample training batch fed into the YOLOv8 model, featuring diverse pavement conditions, road markings and vegetation which helps the model generalize crack detection across various real-world scenarios. Figure 8 presents the ground truth annotations for the validation batch, providing the ideal targets for the model to detect. Figure 9 displays the model's predictions on the same validation batch, successfully identifying most cracks and assigning confidence scores for example 0.7 and 0.8 to each detected bounding box, demonstrating the model's practical effectiveness in real-time crack detection.

7.4 Real-Time Detection using Anaconda Prompt

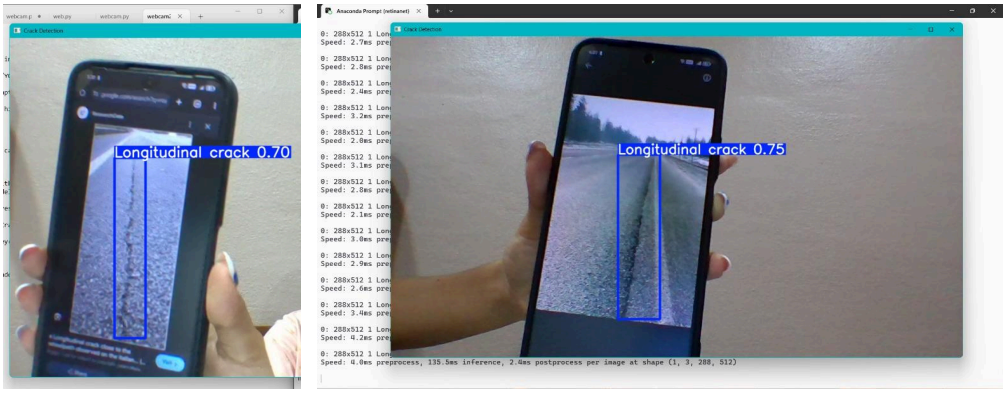

Figure 10: Real-Time Model Validation via Anaconda Prompt

The best.pt model was deployed in an Anaconda environment for live testing using a webcam. The real-time output, shown in Figure 10, demonstrates longitudinal crack detection with confidence scores between 0.70 and 0.75. This confirms the findings of Nguyen et al. (2022), who highlight the need for real-time robustness in practical inspection tasks.

7.5 ROS 1 and Gazebo Simulation

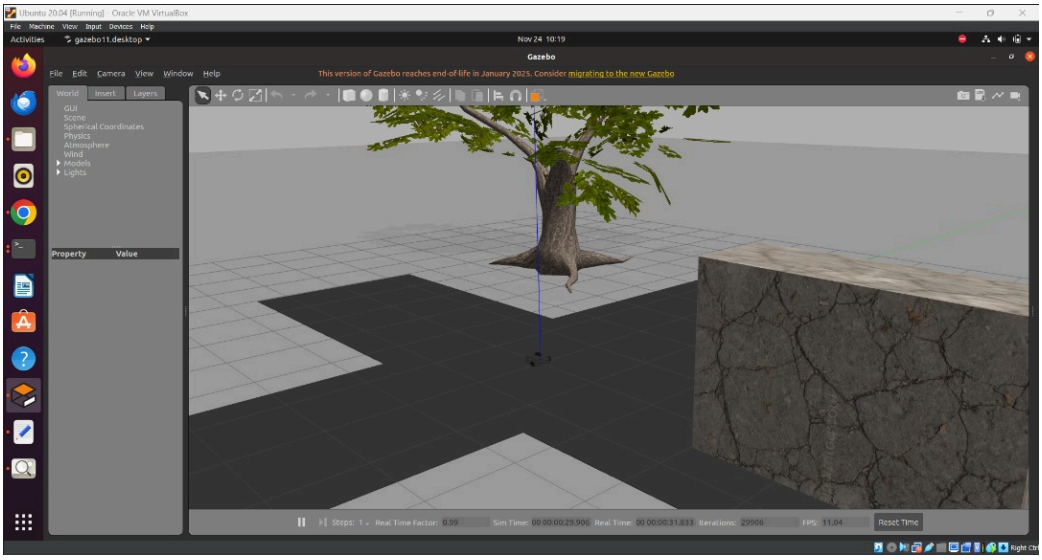

Figure 11: Gazebo Simulation World

A custom Gazebo environment was designed to simulate cracked surfaces, obstacles and natural lighting effects. This environment features a textured road surface, obstacles such as a tree and a cube and the simulated TurtleBot3 Waffle, enabling validation of both autonomous navigation and crack detection. Figure 11 shows the environment used during testing. Camera feeds from the simulated robot are streamed to ROS where YOLOv8 performs inference.

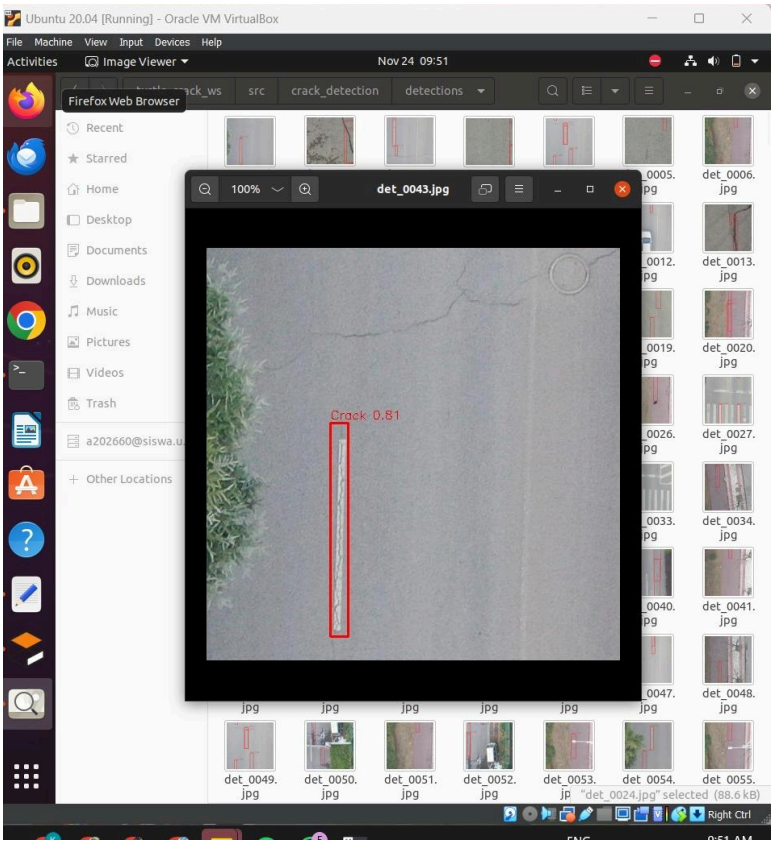

Figure 12: Real-Time Crack Detection in Ubuntu

As illustrated in Figure 12, the output from the deep learning model shows a crack successfully detected on a road segment with a high confidence score of 0.81. This visualization confirms the model's ability to accurately identify and localize pavement damage within the simulated environment.

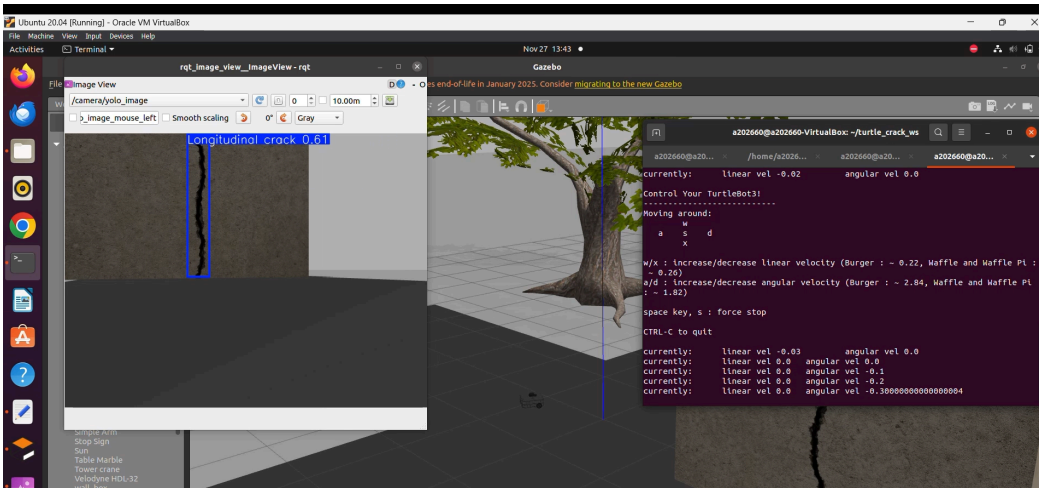

Figure 13: Real-Time Simulation Detection

Finally, Figure 13 demonstrates real-time crack detection inside the Gazebo world using the full ROS 1 pipeline. The YOLOv8 model, deployed on the TurtleBot3 Waffle Pi, successfully detects and labels the longitudinal crack in the simulated environment, achieving high detection accuracy. This confirms seamless integration between perception, simulation and inference, as recommended by Kaveh et al. (2024).

A full simulation video is available in this Google Drive link: https://drive.google.com/drive/folders/18_Ry8Izgs1xATEp1EAL7j1plE87wcBR7?usp=sharing

8.0 Conclusion

This project successfully developed an autonomous crack detection system that integrates deep learning, mobile robotics and simulation-based validation. By training the YOLOv8 model on a cleaned and augmented crack dataset, the system achieves high precision and recall in detecting longitudinal, transverse and alligator cracks across diverse pavement conditions. Deployment in real-time environments using Anaconda Prompt and webcam input demonstrated the model's robustness against varying lighting, camera angles and surface textures.

Integration with ROS 1 Noetic and Gazebo simulation allowed the TurtleBot3 Waffle Pi to navigate autonomously, detect cracks and label defects in real time without

human intervention. The simulation environment ensured safe and repeatable testing, mitigating the risks associated with manual inspection. Overall, the project addresses key limitations of traditional inspection methods, including human error, labor intensity and safety hazards while providing a scalable and reliable autonomous inspection pipeline.

The results indicate that combining deep learning with robotic platforms can significantly enhance infrastructure maintenance workflows, offering a consistent, efficient and high-precision solution suitable for industrial deployment. This work lays a foundation for further research into fully autonomous inspection systems capable of operating in complex and dynamic environments.

9.0 References

[1.] Google Drive repository. (2025). Project simulation videos for autonomous crack detection using TurtleBot3 and YOLOv8. https://drive.google.com/drive/folders/18_Ry8Izgs1xATEp1EAL7j1plE87wcBR7?usp=sharing

[2.] UAPD Pavement Distress Dataset. (n.d.). Dataset repository for pavement crack images. GitHub. https://github.com/tantantetetao/UAPD-Pavement-Distress-Dataset/blob/main/README.md

[3.] Fan, Z. W., Y.; Lu, Z. 2022. Deep learning-based crack detection for industrial surfaces. IEEE Access 10(https://ieeexplore.ieee.org/document/9936270

[4.] Huang, Y. L., J.; Kim, H. 2022. Autonomous mobile robot inspection system using ROS2 and deep learning. Robotics and Autonomous Systems https://doi.org/10.1016/j.robot.2022.104278

[5.] Kaveh, H. A., R. 2024. Recent Advances in Crack Detection Technologies for Structures: A Survey of 2022–2023 Literature. Frontiers in Built Environment 10(https://doi.org/10.3389/fbuil.2024.1321634

[6.] Liu, Y., Et Al. 2023. A Rapid Bridge Crack Detection Method Based on Deep Learning. Applied Sciences (Appl. Sci.) 13(17): https://doi.org/10.3390/app13179878

[7.] Nguyen, S. D. T., T.S.; Le, V.P.; Piran, M.J. 2022. Deep Learning-Based Crack Detection: A Survey. International Journal of Pavement Research and Technology https://doi.org/10.1007/s42947-022-00172-z

[8.] Zhang, J. S., S.; Song, W.; Li, Y.; Teng, Q. 2024. A Novel Convolutional Neural Network for Enhancing the Continuity of Pavement Crack Detection (CPCDNet). Scientific Reports 14(https://www.nature.com/articles/s41598-024-81119-1

[9.] Zhang, J. X., H.; Li, P.; Zhang, K.; Hong, W.; Guo, R. 2024. A Pavement Crack Detection Method via Deep Learning and a Binocular-Vision-Based Unmanned Aerial Vehicle. Applied Sciences (Appl. Sci.) 14(5): 1778. https://doi.org/10.3390/app14051778

[10.] Zhang, Q. C., S.; Wu, Y.; Ji, Z.; Yan, F.; Huang, S. 2024. Improved U-net network asphalt pavement crack detection method. PLoS ONE https://doi.org/10.1371/journal.pone.0300679